

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

File in C

Perché i file?

Finora i dati entravano dalla **tastiera** (`scanf`) e uscivano sul **terminale** (`printf`).

- Con `./prog > dati.dat` (lezione 12) possiamo già mandare l'output in un file, ma **uno solo**.
- Un programma vero spesso deve **leggere** dati di misura (centinaia di righe!) e **scrivere** più file di risultati.
- Un file è una sequenza di byte salvata su disco: **sopravvive** alla fine del programma.
- In C si lavora con i file tramite le funzioni di `<stdio.h>` : le stesse dove vivono `printf` e `scanf` .

Tastiera / terminale	File
`printf(...)`	`fprintf(fp, ...)`
`scanf(...)`	`fscanf(fp, ...)`
sempre disponibili	vanno **aperti** con `fopen`
–	vanno **chiusi** con `fclose`

La `f` iniziale sta per *file*: si usano come `printf` e `scanf` , con in più l'indicazione del file su cui lavorare.

Aprire un file: fopen

```
1   FILE *fp;
2   fp = fopen("nome_del_file", "modo");
```

- `FILE` è un tipo definito in `<stdio.h>` che contiene le informazioni sul file aperto.
- `fopen` restituisce un `FILE *`: un **puntatore** a file (cosa sia un puntatore lo vedremo più avanti). Per ora: è la "maniglia" con cui indichiamo il file.
- Il nome del file è relativo alla **cartella da cui lanciate il programma**.

Modo	Significato	Se il file non esiste	Se il file esiste
`"r"`	lettura (*read*)	errore	si legge dall'inizio
`"w"`	scrittura (*write*)	viene creato	viene **svuotato**!
`"a"`	aggiunta (*append*)	viene creato	si scrive in fondo

Attenzione a `"w"`: aprire in scrittura un file esistente ne **cancella** il contenuto, senza chiedere conferma.

Controllare l'apertura

Se `fopen` fallisce (file inesistente, cartella sbagliata, permessi…) restituisce `NULL`.

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("misure.dat", "r");
    if (fp == NULL) {
        perror("misure.dat");
        return 1;
    }
    printf("File aperto\n");
    fclose(fp);
    return 0;
}
```

```
$ gcc -Wall -Wextra -o apri apri.c
$ ./apri
misure.dat: No such file or directory
$ echo $?
1
```

- `perror` stampa il testo indicato seguito dalla **causa** dell'errore.
- In alternativa: `fprintf(stderr, "Errore: ...\n");` scrive sullo *standard error*.
- `return 1`: il programma termina segnalando un errore (lezione 8).
- Usare un `fp` uguale a `NULL` fa andare in **crash** il programma: controllate **sempre**!

Scrivere su file: fprintf

```c
#include <stdio.h>
#include <math.h>

int main() {
    const double A     = 0.10;        // [m]
    const double gamma = 0.5;         // [1/s]
    const double omega = 2.0 * M_PI;  // [rad/s]
    const double dt    = 0.01;        // [s]

    FILE *fp = fopen("oscillatore.dat", "w");
    if (fp == NULL) {
        perror("oscillatore.dat");
        return 1;
    }
    fprintf(fp, "# t[s]  x[m]\n");
    for (int i = 0; i <= 500; i++) {
        double t = i * dt;
        double x = A * exp(-gamma * t) * cos(omega * t);
        fprintf(fp, "%.2f %.6f\n", t, x);
    }
    fclose(fp);
    return 0;
}
```

Oscillatore smorzato: $x(t) = A\,e^{-\gamma t}\cos(\omega t)$

```
$ gcc -Wall -Wextra -o osc osc.c -lm
$ ./osc
$ head -4 oscillatore.dat
# t[s]  x[m]
0.00 0.100000
0.01 0.099305
0.02 0.098224
```

- `fprintf` funziona come `printf`, con il file come **primo** argomento.
- Sul terminale non compare nulla: tutto l'output è nel file.
- `fclose` chiude il file: i dati vengono effettivamente scritti su disco.

Aggiungere in fondo: modo "a"

```c
#include <stdio.h>

int main() {
    double T;   // [K]
    printf("Temperatura del campione [K]: ");
    scanf("%lf", &T);

    FILE *fp = fopen("registro.txt", "a");
    if (fp == NULL) {
        perror("registro.txt");
        return 1;
    }
    fprintf(fp, "%.1f\n", T);
    fclose(fp);
    return 0;
}
```

```
$ ./registro
Temperatura del campione [K]: 293.4
$ ./registro
Temperatura del campione [K]: 295.1
$ cat registro.txt
293.4
295.1
```

- Con `"a"` ogni esecuzione **aggiunge** una riga: utile per un registro di misure.
- Con `"w"` il file conterrebbe solo l'**ultima** misura: ogni apertura lo svuota.

Leggere da file: fscanf

File `decadimento.dat`: conteggi di una sorgente radioattiva in finestre di 10 s.

```
$ cat decadimento.dat
0 1011
10 803
20 592
...
90 104
```

```c
FILE *fp = fopen("decadimento.dat", "r");
double t;       // [s]
int    conti;
int    r;
r = fscanf(fp, "%lf %d", &t, &conti);
printf("r = %d, t = %.0f, conti = %d\n",
       r, t, conti);
```

```
r = 2, t = 0, conti = 1011
```

- `fscanf` funziona come `scanf` (lezione 13): stessi specificatori, stessa `&`.
- Ogni chiamata legge **a partire da dove si era fermata** la precedente.
- Gli spazi e gli "a capo" nel file contano solo come **separatori**.
- `fscanf` **restituisce** il numero di valori letti con successo: qui `2`.

Come per `scanf`: `%lf` per i `double`, `%f` per i `float`.
Con `printf` invece `%f` va bene per entrambi.

Il valore restituito da fscanf

```c
// due_righe.dat contiene:  0 1011
//                         10 803
printf("%d\n", fscanf(fp, "%lf %d", &t, &conti));
printf("%d\n", fscanf(fp, "%lf %d", &t, &conti));
printf("%d\n", fscanf(fp, "%lf %d", &t, &conti));
```

```
2
2
-1
```

- `2` : letti entrambi i valori.
- `-1` : il file è **finito**. Il valore `-1` ha un nome: la costante `EOF` (*End Of File*) di `<stdio.h>`.
- Un numero minore di `2` (anche `0`) indica un dato **non leggibile**: per esempio una parola dove ci si aspetta un numero.

Leggere fino alla fine del file

```c
double t;      // [s]
int    conti;
int    totale = 0;
while (fscanf(fp, "%lf %d", &t, &conti) == 2) {
    totale += conti;
}
fclose(fp);
printf("Conteggi totali: %d\n", totale);
```

```
Conteggi totali: 4158
```

Non serve sapere **in anticipo** quante righe ci sono: il ciclo continua finché `fscanf` riesce a leggere una riga completa.

Leggere le colonne in array

```c
#include <stdio.h>
#include <math.h>
#define NMAX 1000

int main() {
    double t[NMAX];   // [s]
    int    N[NMAX];   // conteggi
    int    n = 0;     // righe lette
    FILE *fp = fopen("decadimento.dat", "r");
    if (fp == NULL) {
        perror("decadimento.dat");
        return 1;
    }
    while (n < NMAX &&
           fscanf(fp, "%lf %d", &t[n], &N[n]) == 2) {
        n++;
    }
    fclose(fp);
    double tau = (t[n-1] - t[0])
                 / log((double) N[0] / N[n-1]);
    printf("Lette %d righe\n", n);
    printf("Vita media stimata: %.1f s\n", tau);
```

```
$ gcc -Wall -Wextra -o vita vita.c -lm
$ ./vita
Lette 10 righe
Vita media stimata: 39.6 s
```

- L'array è dimensionato per il caso **peggiore** (`NMAX`); `n` conta le righe davvero lette.
- `n < NMAX` evita di scrivere **oltre la fine** dell'array se il file è troppo lungo.
- Da $N(t) = N_0\, e^{-t/\tau}$ con il primo e l'ultimo punto: $\tau = \dfrac{t_{n-1} - t_0}{\ln(N_0/N_{n-1})}$.
- Ora i dati sono in memoria: potete usare tutto ciò che avete visto nelle lezioni 18 e 20.

Attenzione alle intestazioni

```
$ cat con_intestazione.dat
# t[s] conteggi
0 1011
10 803
```

```
printf("%d\n", fscanf(fp, "%lf %d", &t, &conti));
```

```
0
```

- `fscanf` trova `#` dove si aspetta un numero: legge **0** valori e si ferma lì.
- Il ciclo `while (... == 2)` non parte nemmeno: il programma "legge" zero righe senza lamentarsi.

- `np.loadtxt` ignora le righe che iniziano con `#`; `fscanf` **no**.
- Regola pratica: i file che un programma C deve rileggere li scriviamo **senza** intestazione (oppure sappiamo esattamente quante righe saltare).
- Controllate sempre **quante righe** avete letto: stampare `n` è il test più semplice.

Leggere e saltare righe di testo richiede gli strumenti sulle stringhe: li vedremo più avanti.

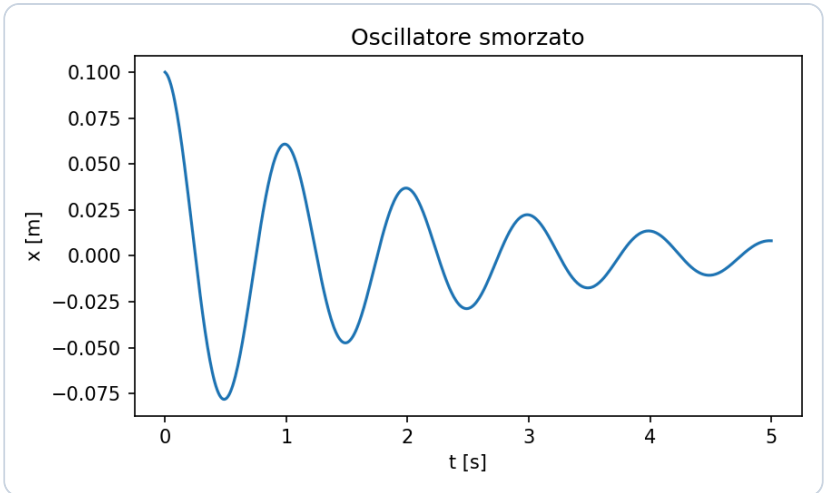
Dal C a Python

Il file `oscillatore.dat` scritto in C è pronto per `np.loadtxt` (lezione 9).

```python
import numpy as np
import matplotlib.pyplot as plt

t, x = np.loadtxt('oscillatore.dat', unpack=True)

plt.plot(t, x)
plt.xlabel('t [s]')
plt.ylabel('x [m]')
plt.title('Oscillatore smorzato')
plt.savefig('oscillatore.png')
```



- La riga `# t[s] x[m]` viene ignorata da `loadtxt`.
- Il C **calcola**, Python **disegna**: è lo schema che userete in laboratorio.

stdin, stdout e stderr

Tre "file" sono **già aperti** all'avvio di ogni programma C.

Nome	Che cos'è	Di solito è
`stdin`	*standard input*	la tastiera
`stdout`	*standard output*	il terminale
`stderr`	*standard error*	il terminale

- Sono di tipo `FILE *`, come quelli restituiti da `fopen`.
- `printf(...)` è equivalente a `fprintf(stdout, ...)`.
- `scanf(...)` è equivalente a `fscanf(stdin, ...)`.

```c
fprintf(stdout, "%.2f %.6f\n", t, x);
fprintf(stderr, "Attenzione: dt troppo grande\n");
```

- `>` redirige **solo** `stdout`; `2>` redirige `stderr`.
- I messaggi di errore su `stderr` restano visibili anche quando l'output è rediretto in un file, e non "sporcano" i dati.

```
$ ./prog > dati.dat 2> errori.txt
```

Cenni ai file binari

```c
#include <stdio.h>
#include <math.h>
#define N 1000

int main() {
    double x[N];
    for (int i = 0; i < N; i++) x[i] = sin(0.01 * i);

    FILE *ft = fopen("dati.txt", "w");
    for (int i = 0; i < N; i++) {
        fprintf(ft, "%.17g\n", x[i]);
    }
    fclose(ft);

    FILE *fb = fopen("dati.bin", "wb");
    fwrite(x, sizeof(double), N, fb);
    fclose(fb);
    return 0;
}
```

(Per brevità qui non controlliamo `NULL`: nei vostri programmi fatelo sempre.)

```
$ wc -c dati.txt dati.bin
   20337 dati.txt
    8000 dati.bin
   28337 total
```

- `wc -c` conta i byte di un file.
- `fwrite(indirizzo, dimensione, quanti, fp)` copia su disco i **byte** della memoria così come sono.
- $1000 \times 8 = 8000$ byte: esattamente `sizeof(double)` per valore.
- Il file di testo con 17 cifre significative è **2.5 volte** più grande.
- Modi `"wb"` e `"rb"`: la `b` sta per *binary*.

Rileggere un file binario: fread

```c
#include <stdio.h>
#define N 1000

int main() {
    double y[N];

    FILE *fb = fopen("dati.bin", "rb");
    if (fb == NULL) {
        perror("dati.bin");
        return 1;
    }
    int letti = fread(y, sizeof(double), N, fb);
    fclose(fb);

    printf("Letti %d valori\n", letti);
    printf("y[100] = %.17g\n", y[100]);
    return 0;
}
```

```
Letti 1000 valori
y[100] = 0.8414709848078965
```

	Testo	Binario
Leggibile con `cat`, editor	sì	no
Leggibile da `np.loadtxt`	sì	no
Dimensione	più grande	compatta
Precisione	quella stampata	**esatta**
Velocità	lenta	veloce

In questo corso useremo quasi sempre file di **testo**.

Errori tipici con i file

Errore	Cosa succede
Non controllare se `fopen` ha restituito `NULL`	Crash alla prima `fprintf` / `fscanf` (*Segmentation fault*)
Lanciare il programma da un'altra cartella	Il file "non esiste": il percorso è relativo alla cartella corrente
Aprire con `"w"` un file di dati da conservare	Il contenuto precedente è **perso**
`%f` invece di `%lf` in `fscanf` per un `double`	Valori senza senso; warning `format '%f' expects argument of type 'float *'`
Dimenticare `&` in `fscanf`	Crash o valori sbagliati; warning `-Wformat`
Intestazione con `#` nel file da leggere	`fscanf` restituisce `0` e non legge nulla
Ciclo di lettura che ignora il valore restituito da `fscanf`	Ciclo infinito o dati ripetuti/sbagliati
Dimenticare `fclose`	Il file può restare incompleto

Mettetevi alla prova

Il file `dati.dat` contiene le tre righe sotto il codice. Cosa succede? Pensateci **prima** di andare avanti.

```c
FILE *fp = fopen("dati.dat", "r");
double x, y;
int n = 0;
while (fscanf(fp, "%lf %lf", &x, &y) == 2) {
    n++;
}
printf("%d\n", n);
```

```
1.0 2.0
3.0 abc
5.0 6.0
```

Domanda	Risposta
Cosa stampa?	`1`
Perché?	Alla seconda riga `fscanf` trova `abc`: restituisce `1` e il ciclo si ferma
E se il file non esiste?	`fp` vale `NULL`: crash, perché manca il controllo
E se aprissimo con `"w"`?	Il file viene **svuotato**; `n` vale `0`
E con `"a"` e poi una `fprintf`?	Il testo va **in fondo** al file